

7. Reverse Integer

7. Reverse Integer

Solved 

Medium

 Topics

 Companies

Given a signed 32-bit integer `x`, return `x` with its digits reversed. If reversing `x` causes the value to go outside the signed 32-bit integer range $[-2^{31}, 2^{31} - 1]$, then return `0`.

Assume the environment does not allow you to store 64-bit integers (signed or unsigned).

Example 1:

Input: `x = 123`

Output: `321`

Example 2:

Input: `x = -123`

Output: `-321`

Example 3:

Input: `x = 120`

Output: `21`

Basic Intuition:

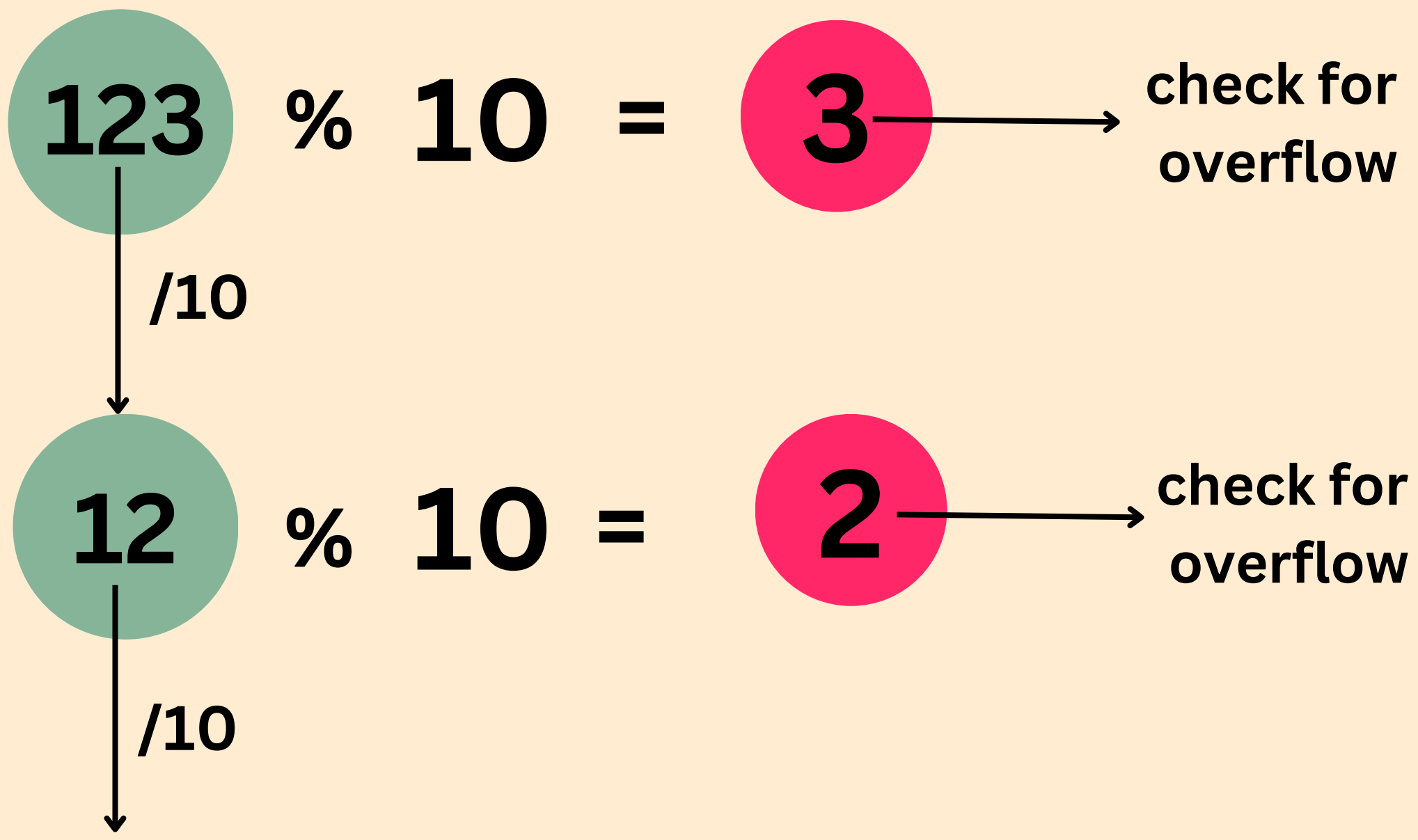
- Start with the given number.
- Take its last digit.
- Check if adding that digit to the reversed result will cause an overflow (make the number too big).
- If it's safe, add the digit to the reversed result.
- Remove the last digit from the given number.
- Repeat these steps until the original number becomes zero.

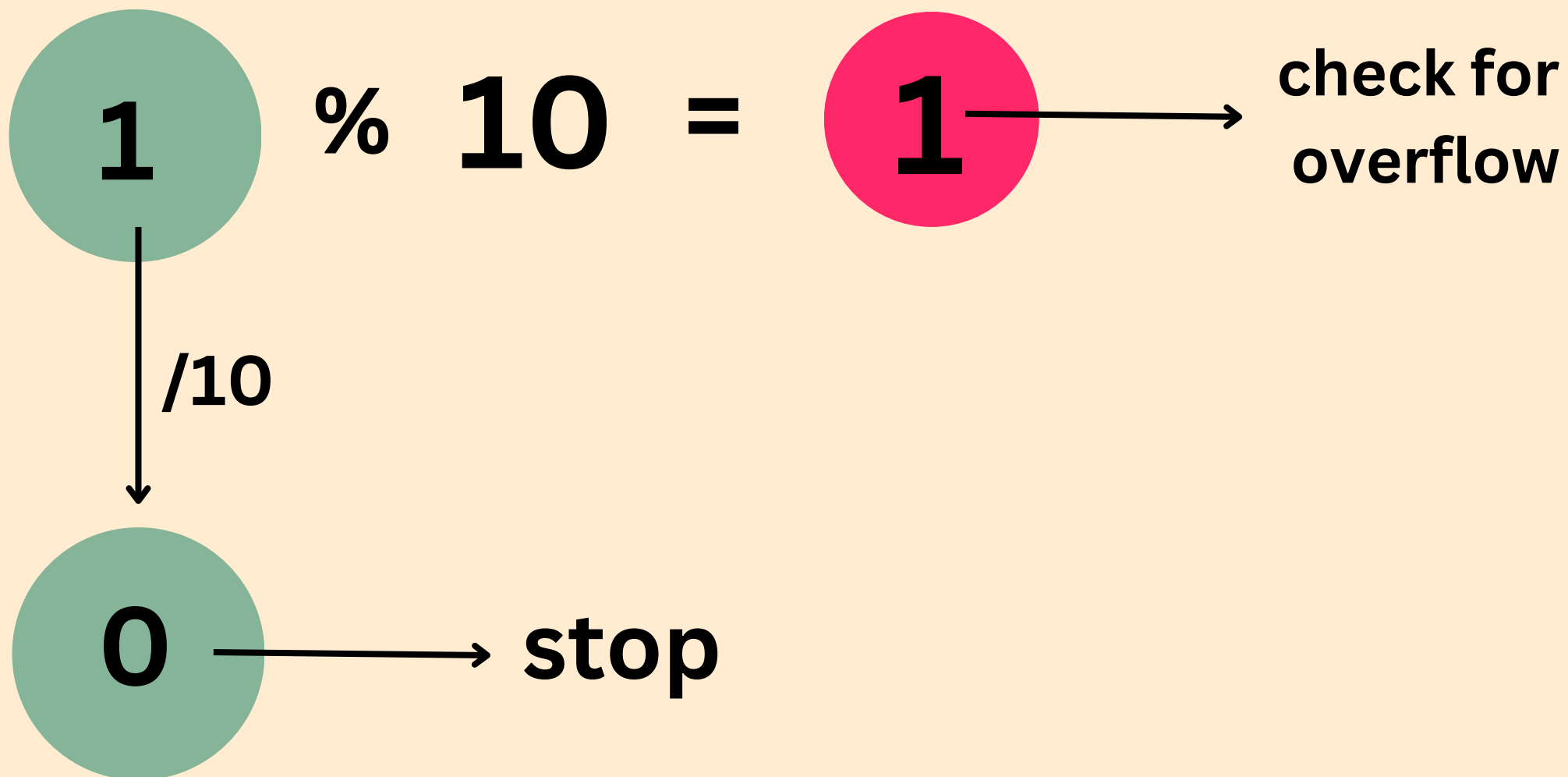
understand from examples:

Ex: 123

```
int rev_no = 0
```

x = 123





Ex: 123

x = 1534236469

reverse

rev = 9646324351

rev = 9646324351



output: 0

Because this number exceeds the range of a 32-bit signed integer

Approach 1:-

C++   Auto

```
1  class Solution {
2  public:
3      int reverse(int x) {
4          int rev = 0;
5          while (x != 0) {
6              int last_digit = x % 10;
7              x = x / 10;
8              //check for overflow
9              if (rev > INT_MAX / 10 || (rev == INT_MAX / 10 && last_digit > 7)) {
10                 return 0;
11             }
12             //check for overflow
13             if (rev < INT_MIN / 10 || (rev == INT_MIN / 10 && last_digit < -8)) {
14                 return 0;
15             }
16
17             rev = (rev * 10) + last_digit;
18         }
19         return rev;
20     }
21 };
```

Accepted Runtime: 3 ms

• Case 1 • Case 2 • Case 3

Input

x =
123

Output

321

Expected

321

 [Contribute a testcase](#)

Accepted

 Manish Kumar submitted at Feb 10, 2024 22:41

 Editorial

 Solution

 Runtime

2 ms

Beats 39.62% of users with C++

- **Time complexity:** $O(\log(x))$
- **Space complexity:** $O(1)$

Approach 2:

- **Initialize Result Variable.**
- **Iterative Reversal.**
- **Continue Until Original Number is Zero.**
- **Handle Overflow.**

</> Code

C++   Auto

```
1  class Solution {
2  public:
3      int reverse(int x) {
4          long ans = 0;
5          while(x != 0)
6          {
7              int last_digit = x % 10;
8              ans = ans*10 + last_digit;
9              x = x/10;
10         }
11         if(ans > INT_MAX || ans < INT_MIN)
12         {
13             return 0;
14         }
15         return static_cast<int>(ans);
16     }
17 };
```

Accepted Runtime: 0 ms

• Case 1 • Case 2 • Case 3

Input

x =
123

Output

321

Expected

321

Accepted

 Manish Kumar submitted at Feb 10, 2024 22:53

 Editorial

 Solution

 Runtime

0 ms

 Beats 100.00% of users with C++

- Time complexity: $O(\log(x))$
- Space complexity: $O(1)$



follow me on:



<https://www.linkedin.com/in/manish-kumar-689905213/>



manishkumar_978